

title: Контракт Premium Вложения — Комбинированные Вложения (Combined)
description: Полная спецификация комбинированных вложений
version: 1.0.0
date: 2026-06-14
author: Koda (NLP-Core-Team)
status: complete
audience: both
tags:

- premium
 - combined
 - contract
 - specification
- related_docs:
- SUMMARY_DOCS/premium/PREMIUM_ATTACHMENTS_OVERVIEW.md
 - messenger/src/types/attachments.ts

PREMIUM CONTRACT: КОМБИНИРОВАННЫЕ ВЛОЖЕНИЯ (COMBINED)

Версия: 1.0.0
Дата: 2026-06-14
Статус: ☒ Complete
Доступ:  Только "Шейх"

1. ОБЗОР

1.1 Назначение

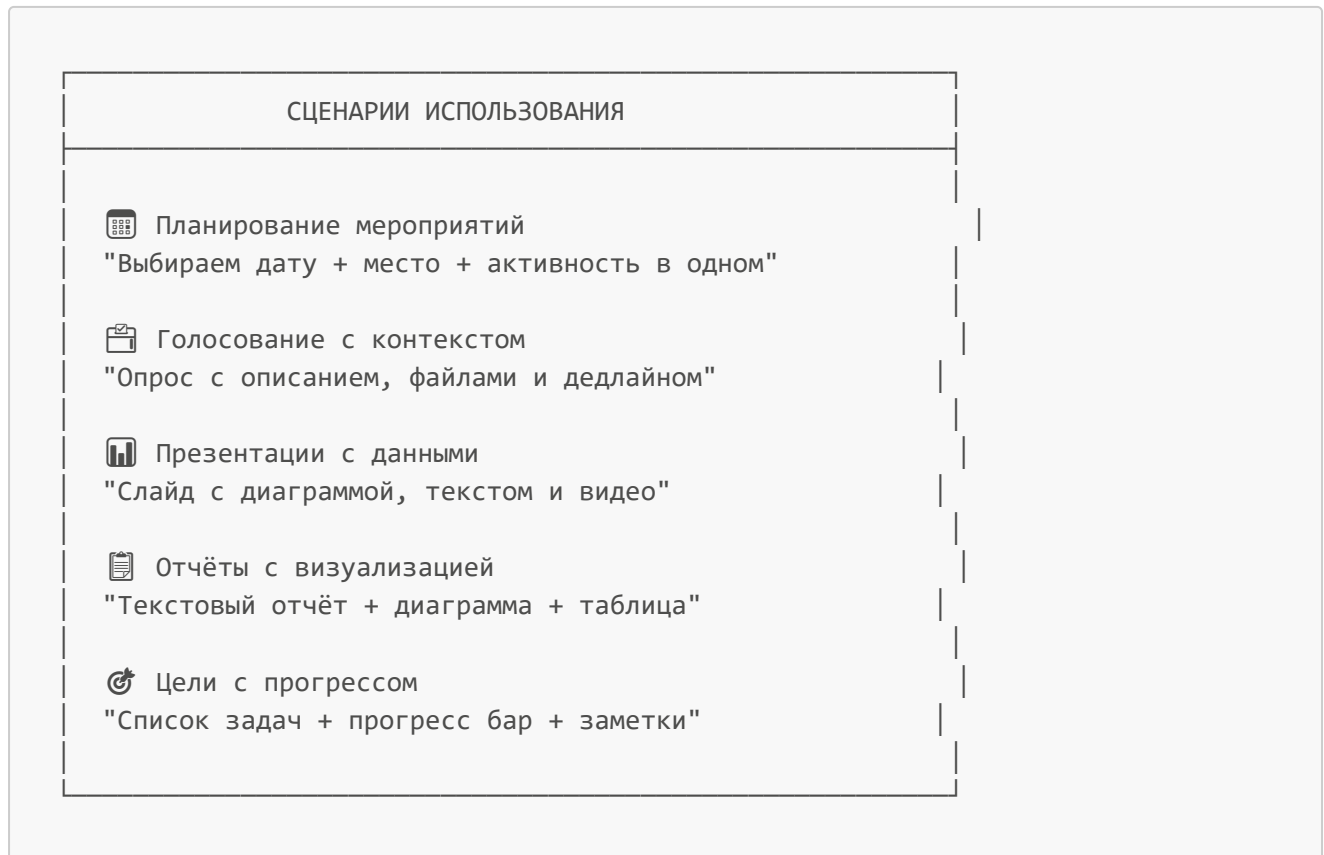
Комбинированные вложения (Combined) — это premium вложение, объединяющее несколько типов вложений в одном интерфейсе. Позволяет создавать сложные интерактивные сообщения для планирования, голосования, презентаций и отчётов.

1.2 Статус доступа

Параметр	Значение
Доступ	 Premium (Шейх)
Категория	Продуктивность / Интерактив
Сложность	 Высокая
Время реализации	4-6 дней

Параметр	Значение
Уникальность	💡 Эксклюзив Balloo

1.3 Use Cases



2. ТИПЫ ДАННЫХ

2.1 Основной интерфейс

```
/**
 * Тип вложения
 */
export type CombinedAttachmentType = 'combined';

/**
 * Пресеты комбинаций
 */
export type CombinedPreset =
  | 'event_planner' // Событие + Опрос + Список
  | 'poll_with_context' // Опрос + Текст + Файл
  | 'presentation' // Текст + Изображение + Диаграмма
  | 'report' // Текст + Диаграмма + Таблица
  | 'goal_tracker' // Список + Прогресс + Заметки
  | 'brainstorm' // Доска + Голосование + Заметки
  | 'custom'; // Пользовательская комбинация
```

```

/**
 * Комбинированное вложение
 */
export interface CombinedAttachment {
  type: 'combined';
  attachmentId: string;
  combinedId: string;

  // Пресет
  preset: CombinedPreset;
  template?: CombinedTemplate;

  // Компоненты
  components: CombinedComponent[];
  layout: ComponentLayout;

  // Взаимосвязи
  relations: ComponentRelation[];

  // Результаты/Действия
  actions: CombinedAction[];
  results?: CombinedResults;

  // Настройки
  settings: CombinedSettings;

  // Участники
  createdBy: string;
  contributors: string[];
  viewers: string[];

  // Статус
  status: 'draft' | 'active' | 'completed' | 'archived';

  // Метаданные
  metadata: Record<string, any>;

  createdAt: number;
  updatedAt: number;
  completedAt?: number;
}

/**
 * Компонент в комбинации
 */
export interface CombinedComponent {
  id: string;
  order: number;

  // Тип вложения
  attachmentType: AttachmentType;

  // Данные вложения

```

```

data: any;

// Конфигурация отображения
display: DisplayConfig;

// Взаимодействие
interaction: InteractionConfig;

// Условия видимости
visibility?: VisibilityCondition[];

// Связи с другими компонентами
relations?: ComponentRelation[];
}

/**
 * Конфигурация отображения
 */
export interface DisplayConfig {
  position: {
    row: number;
    column: number;
    span?: number; // Для grid layout
  };
  size: {
    width: 'full' | 'half' | 'third' | 'quarter';
    height: 'auto' | 'fixed';
    fixedHeight?: number;
  };
  style: {
    theme: 'light' | 'dark' | 'auto';
    accentColor?: string;
    showBorder: boolean;
    showTitle: boolean;
  };
  animations: {
    entrance: 'fade' | 'slide' | 'scale' | 'none';
    duration: number;
  };
}

/**
 * Конфигурация взаимодействия
 */
export interface InteractionConfig {
  clickable: boolean;
  expandable: boolean;
  collapsible: boolean;
  defaultState: 'expanded' | 'collapsed';

  // Для интерактивных компонентов
  allowMultipleInteractions: boolean;
  requireConfirmation: boolean;
}

```

```

    showFeedback: boolean;
}

/**
 * Условия видимости
 */
export interface VisibilityCondition {
    type: 'user_role' | 'user_id' | 'time' | 'interaction';

    // Для user_role
    requiredRole?: 'owner' | 'editor' | 'viewer';

    // Для user_id
    allowedUsers?: string[];
    deniedUsers?: string[];

    // Для time
    visibleFrom?: number;
    visibleUntil?: number;

    // Для interaction
    visibleAfter?: {
        componentId: string;
        action: 'viewed' | 'interacted' | 'completed';
    };
}

/**
 * Связи между компонентами
 */
export interface ComponentRelation {
    fromComponent: string;
    toComponent: string;
    type: 'depends_on' | 'updates' | 'filters' | 'triggers';

    // Логика
    condition?: RelationCondition;
    transform?: DataTransform;
}

export interface RelationCondition {
    operator: 'equals' | 'not_equals' | 'greater_than' | 'less_than' |
    'contains';
    field: string;
    value: any;
}

export interface DataTransform {
    type: 'filter' | 'map' | 'aggregate' | 'sort';
    config: any;
}

/**

```

```

* Действия
*/
export interface CombinedAction {
  id: string;
  name: string;
  icon: string;

  // Триггер
  trigger: 'button' | 'auto' | 'schedule';

  // Условия
  conditions?: ActionCondition[];

  // Выполнение
  execution: ActionExecution;

  // Уведомления
  notifications: NotificationConfig[];
}

export interface ActionCondition {
  componentId: string;
  field: string;
  operator: string;
  value: any;
}

export interface ActionExecution {
  type: 'api' | 'webhook' | 'notification' | 'update_component';

  // Для api
  endpoint?: string;
  method?: 'GET' | 'POST' | 'PUT' | 'DELETE';
  payload?: any;

  // Для update_component
  targetComponent?: string;
  updateData?: any;
}

/**
* Результаты
*/
export interface CombinedResults {
  summary: ResultSummary;
  componentResults: Record<string, ComponentResult>;
  interactions: InteractionLog[];
  exports: ExportEntry[];
}

export interface ResultSummary {
  totalInteractions: number;
  uniqueParticipants: number;
}

```

```

    completionRate: number;
    averageTimeSpent: number;
    keyMetrics: Record<string, any>;
}

export interface ComponentResult {
    componentId: string;
    responses: number;
    data: any;
    statistics?: any;
}

export interface InteractionLog {
    userId: string;
    componentId: string;
    action: string;
    data: any;
    timestamp: number;
}

export interface ExportEntry {
    format: 'pdf' | 'docx' | 'xlsx' | 'json' | 'csv';
    url: string;
    createdAt: number;
    expiresAt: number;
}

/**
 * Настройки
 */
export interface CombinedSettings {
    allowEditing: boolean;
    allowComments: boolean;
    showAnalytics: boolean;
    requireAllComponents: boolean;
    submissionMode: 'single' | 'multiple';
    expiration?: {
        expiresAt: number;
        action: 'close' | 'delete' | 'archive';
    };
    notifications: {
        onInteraction: boolean;
        onCompletion: boolean;
        onExpiry: boolean;
    };
}

```

2.2 Пресеты

```

/**
 * Пресет: Планировщик событий
 */
export interface EventPlannerPreset extends CombinedAttachment {
  preset: 'event_planner';
  components: [
    EventDetailsComponent,      // Текст + Дата + Место
    DatePollComponent,          // Опрос для выбора даты
    LocationPollComponent,      // Опрос для выбора места
    ActivityListComponent,      // Список активностей
    BudgetComponent             // Текст + Диаграмма
  ];
}

export interface EventDetailsComponent extends CombinedComponent {
  attachmentType: 'note';
  data: {
    title: string;
    description: string;
    proposedDates: number[];
    proposedLocations: string[];
  };
}

export interface DatePollComponent extends CombinedComponent {
  attachmentType: 'poll';
  data: {
    question: 'Выберите удобную дату';
    options: PollOption[];
    multipleChoice: false;
  };
}

/**
 * Пресет: Опрос с контекстом
 */
export interface PollWithContextPreset extends CombinedAttachment {
  preset: 'poll_with_context';
  components: [
    ContextTextComponent,      // Описание
    ContextFileComponent,      // Файл/изображение
    PollComponent,             // Сам опрос
    DeadlineComponent          // Дедлайн
  ];
}

/**
 * Пресет: Презентация
 */
export interface PresentationPreset extends CombinedAttachment {
  preset: 'presentation';
  components: [

```



```

    SlideComponent[]          // Массив слайдов
  ];
}

export interface SlideComponent extends CombinedComponent {
  attachmentType: 'combined';
  data: {
    slideNumber: number;
    title: string;
    elements: SlideElement[];
  };
}

export interface SlideElement {
  type: 'text' | 'image' | 'chart' | 'video' | 'list';
  content: any;
  position: { x: number; y: number; width: number; height: number };
}

/**
 * Пресет: Отчёт
 */
export interface ReportPreset extends CombinedAttachment {
  preset: 'report';
  components: [
    ReportHeaderComponent,    // Заголовок + мета
    SummaryTextComponent,     // Краткое описание
    ChartComponent,           // Диаграмма
    TableComponent,           // Таблица с данными
    ConclusionComponent        // Выводы
  ];
}

/**
 * Пресет: Трекер целей
 */
export interface GoalTrackerPreset extends CombinedAttachment {
  preset: 'goal_tracker';
  components: [
    GoalDescriptionComponent,  // Описание цели
    TaskListComponent,         // Список задач
    ProgressComponent,         // Прогресс бар
    NotesComponent             // Заметки
  ];
}

/**
 * Пресет: Мозговой штурм
 */
export interface BrainstormPreset extends CombinedAttachment {
  preset: 'brainstorm';
  components: [
    WhiteboardComponent,       // Виртуальная доска

```

```

    IdeaVotingComponent,      // Голосование за идеи
    DiscussionComponent       // Обсуждение
  ];
}

```

2.3 Шаблоны

```

/**
 * Шаблон комбинации
 */
export interface CombinedTemplate {
  id: string;
  name: string;
  description: string;
  category: string;

  // Пресет
  preset: CombinedPreset;

  // Компоненты с дефолтными значениями
  components: CombinedComponent[];

  // Настройки по умолчанию
  defaultSettings: CombinedSettings;

  // Пример использования
  example?: any;

  // Метаданные
  author: string;
  isPublic: boolean;
  usageCount: number;
  rating: number;
  tags: string[];

  createdAt: number;
  updatedAt: number;
}

/**
 * Библиотека шаблонов
 */
export interface TemplateLibrary {
  categories: TemplateCategory[];
  featured: CombinedTemplate[];
  recent: CombinedTemplate[];
  userTemplates: CombinedTemplate[];
}

export interface TemplateCategory {

```

```

id: string;
name: string;
icon: string;
templates: CombinedTemplate[];
count: number;
}

```

3. API SPECIFICATION

3.1 Endpoints

COMBINED API ENDPOINTS		
POST	/api/v1/combined/create	Создать
POST	/api/v1/combined/:id/update	Обновить
POST	/api/v1/combined/:id/interact	Взаимодействие
POST	/api/v1/combined/:id/complete	Завершить
GET	/api/v1/combined/:id	Получить
GET	/api/v1/combined/:id/results	Результаты
GET	/api/v1/combined/:id/export	Экспорт
GET	/api/v1/combined/templates	Шаблоны
POST	/api/v1/combined/templates	Сохранить шаблон
DELETE	/api/v1/combined/:id/archive	Архивировать

3.2 Создать комбинированное вложение

Endpoint: POST /api/v1/combined/create

Headers:

```

Authorization: Bearer <token>
Content-Type: application/json
X-Sheikh-Status: true

```

Request Body (Планировщик событий):

```

{
  "preset": "event_planner",
  "title": "Корпоратив 2026",
  "components": [
    {

```

```

    "id": "comp_details",
    "order": 0,
    "attachmentType": "note",
    "data": {
      "title": "Корпоратив 2026",
      "description": "Планируем ежегодный корпоратив",
      "proposedDates": [1719619200000, 1719705600000, 1719792000000],
      "proposedLocations": ["Ресторан Пушкин", "Лофт Центр", "Загородный
клуб"]
    },
    "display": {
      "position": { "row": 0, "column": 0 },
      "size": { "width": "full", "height": "auto" }
    }
  },
  {
    "id": "comp_date_poll",
    "order": 1,
    "attachmentType": "poll",
    "data": {
      "question": "Выберите удобную дату",
      "options": [
        { "id": "opt1", "text": "29 июня" },
        { "id": "opt2", "text": "30 июня" },
        { "id": "opt3", "text": "1 июля" }
      ],
      "multipleChoice": false
    },
    "display": {
      "position": { "row": 1, "column": 0 },
      "size": { "width": "full", "height": "auto" }
    }
  },
  {
    "id": "comp_location_poll",
    "order": 2,
    "attachmentType": "poll",
    "data": {
      "question": "Выберите место проведения",
      "options": [
        { "id": "loc1", "text": "Ресторан Пушкин" },
        { "id": "loc2", "text": "Лофт Центр" },
        { "id": "loc3", "text": "Загородный клуб" }
      ],
      "multipleChoice": false
    },
    "display": {
      "position": { "row": 2, "column": 0 },
      "size": { "width": "full", "height": "auto" }
    }
  },
  {
    "id": "comp_activities",

```

```

    "order": 3,
    "attachmentType": "list",
    "data": {
      "title": "План мероприятия",
      "items": [
        { "id": "item1", "text": "Сбор гостей", "order": 0 },
        { "id": "item2", "text": "Ужин", "order": 1 },
        { "id": "item3", "text": "Конкурсы", "order": 2 },
        { "id": "item4", "text": "Награждение", "order": 3 }
      ]
    },
    "display": {
      "position": { "row": 3, "column": 0 },
      "size": { "width": "full", "height": "auto" }
    }
  },
  "layout": {
    "type": "vertical",
    "spacing": "comfortable"
  },
  "settings": {
    "allowEditing": true,
    "showAnalytics": true,
    "expiration": {
      "expiresAt": 1719532800000,
      "action": "close"
    }
  }
}

```

Response (Success 201):

```

{
  "success": true,
  "data": {
    "combinedId": "comb_abc123",
    "attachmentId": "att_xyz789",
    "status": "active",
    "components": 4,
    "previewUrl": "https://balloo.app/combined/comb_abc123",
    "message": "Планировщик создан. Ожидание голосов..."
  }
}

```

3.3 Взаимодействие

Endpoint: POST /api/v1/combined/:id/interact

Request Body:

```
{
  "componentId": "comp_date_poll",
  "action": "vote",
  "data": {
    "optionId": "opt2"
  }
}
```

Response:

```
{
  "success": true,
  "data": {
    "combinedId": "comb_abc123",
    "componentId": "comp_date_poll",
    "interactionId": "int_def456",
    "updatedAt": 1718400000000,
    "results": {
      "totalVotes": 15,
      "options": [
        { "id": "opt1", "votes": 3, "percentage": 20 },
        { "id": "opt2", "votes": 8, "percentage": 53 },
        { "id": "opt3", "votes": 4, "percentage": 27 }
      ]
    },
    "triggeredActions": [
      {
        "actionId": "act_update_progress",
        "type": "update_component",
        "targetComponent": "comp_progress"
      }
    ]
  }
}
```

3.4 Результаты

Endpoint: `GET /api/v1/combined/:id/results`

Response:

```
{
  "success": true,
  "data": {
    "combinedId": "comb_abc123",
```

```

"status": "active",
"summary": {
  "totalInteractions": 45,
  "uniqueParticipants": 15,
  "completionRate": 0.87,
  "averageTimeSpent": 125000
},
"componentResults": {
  "comp_date_poll": {
    "responses": 15,
    "data": {
      "winner": "opt2",
      "options": [...]
    }
  },
  "comp_location_poll": {
    "responses": 14,
    "data": {
      "winner": "loc1",
      "options": [...]
    }
  },
  "comp_activities": {
    "responses": 12,
    "data": {
      "completedItems": 2,
      "totalItems": 4,
      "progress": 0.5
    }
  }
},
"interactions": [
  {
    "userId": "user_123",
    "componentId": "comp_date_poll",
    "action": "vote",
    "data": { "optionId": "opt2" },
    "timestamp": 1718400000000
  }
],
"exports": [
  {
    "format": "pdf",
    "url": "https://balloo.app/exports/comb_abc123.pdf",
    "createdAt": 1718400000000,
    "expiresAt": 1718486400000
  }
]
}

```

3.5 Шаблоны

Endpoint: `GET /api/v1/combined/templates`

Query Parameters:

```
?category=events  
&search=party  
&limit=20
```

Response:

```
{  
  "success": true,  
  "data": {  
    "categories": [  
      {  
        "id": "events",  
        "name": "📅 События",  
        "icon": "calendar",  
        "count": 12  
      },  
      {  
        "id": "polls",  
        "name": "🗳 Опросы",  
        "icon": "poll",  
        "count": 8  
      },  
      {  
        "id": "reports",  
        "name": "📊 Отчёты",  
        "icon": "chart",  
        "count": 6  
      }  
    ],  
    "featured": [  
      {  
        "id": "tpl_event_planner",  
        "name": "Планировщик мероприятия",  
        "description": "Полный комплект для организации события",  
        "preset": "event_planner",  
        "usageCount": 1250,  
        "rating": 4.8,  
        "tags": ["event", "planning", "poll"]  
      }  
    ],  
    "userTemplates": [  
      {  
        "id": "tpl_user_1",
```



```
    "name": "Мой шаблон отчёта",
    "preset": "report",
    "isPublic": false,
    "createdAt": 1718300000000
  }
]
}
}
```

4. БИЗНЕС-ЛОГИКА

4.1 Создание комбинации

```
async function createCombinedAttachment(
  userId: string,
  preset: CombinedPreset,
  components: CombinedComponent[],
  settings: CombinedSettings
): Promise<CombinedAttachment> {

  // 1. Проверка статуса Шейх
  const user = await getUserById(userId);
  if (!user.is_sheikh) {
    throw new PremiumRequiredError('combined');
  }

  // 2. Валидация пресета
  const presetConfig = PRESET_CONFIGS[preset];
  if (!presetConfig) {
    throw new InvalidPresetError(preset);
  }

  // 3. Валидация компонентов
  validateComponents(components, presetConfig);

  // 4. Валидация связей
  validateRelations(components);

  // 5. Создание записи
  const combined: CombinedAttachment = {
    type: 'combined',
    attachmentId: generateAttachmentId(),
    combinedId: generateCombinedId(),
    preset,
    components,
    layout: presetConfig.defaultLayout,
    relations: presetConfig.defaultRelations,
    actions: presetConfig.defaultActions,
    settings: {
```

```

        ...presetConfig.defaultSettings,
        ...settings
    },
    createdBy: userId,
    contributors: [userId],
    status: 'active',
    createdAt: Date.now(),
    updatedAt: Date.now()
};

// 6. Сохранение
await saveCombinedAttachment(combined);

// 7. Планирование истечения
if (settings.expiration) {
    await scheduleExpiration(
        combined.combinedId,
        settings.expiration.expiresAt,
        settings.expiration.action
    );
}

return combined;
}

```

4.2 Обработка взаимодействия

```

async function handleInteraction(
    combinedId: string,
    componentId: string,
    userId: string,
    action: string,
    data: any
): Promise<InteractionResult> {

    const combined = await getCombinedAttachment(combinedId);

    // 1. Проверка статуса
    if (combined.status !== 'active') {
        throw new CombinedNotActiveError(combined.status);
    }

    // 2. Проверка компонента
    const component = combined.components.find(c => c.id === componentId);
    if (!component) {
        throw new ComponentNotFoundError(componentId);
    }

    // 3. Проверка видимости
    if (!isVisible(component, userId)) {

```

```

        throw new ComponentNotVisibleError(componentId);
    }

    // 4. Выполнение действия
    const result = await executeComponentAction(
        component,
        action,
        data,
        userId
    );

    // 5. Логирование
    await logInteraction({
        combinedId,
        componentId,
        userId,
        action,
        data,
        timestamp: Date.now()
    });

    // 6. Проверка триггеров
    const triggeredActions = await checkTriggers(combined, componentId, result);

    // 7. Выполнение триггерных действий
    for (const triggerAction of triggeredActions) {
        await executeAction(triggerAction, combined);
    }

    // 8. Проверка завершения
    const isComplete = await checkCompletion(combined, userId);
    if (isComplete && combined.settings.submissionMode === 'single') {
        await markComplete(combinedId, userId);
    }

    // 9. Уведомления
    if (combined.settings.notifications.onInteraction) {
        await notifyCreator(combinedId, {
            userId,
            componentId,
            action
        });
    }

    return {
        success: true,
        result,
        triggeredActions,
        isComplete
    };
}

```

4.3 Экспорт результатов

```
async function exportCombinedResults(
  combinedId: string,
  format: 'pdf' | 'docx' | 'xlsx' | 'json' | 'csv'
): Promise<ExportEntry> {

  const combined = await getCombinedAttachment(combinedId);
  const results = await getCombinedResults(combinedId);

  // 1. Генерация контента
  let content: Buffer;
  let mimeType: string;

  switch (format) {
    case 'pdf':
      content = await generatePDF(combined, results);
      mimeType = 'application/pdf';
      break;
    case 'docx':
      content = await generateDOCX(combined, results);
      mimeType = 'application/vnd.openxmlformats-officedocument.wordprocessingml.document';
      break;
    case 'xlsx':
      content = await generateXLSX(results);
      mimeType = 'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet';
      break;
    case 'json':
      content = Buffer.from(JSON.stringify(results, null, 2));
      mimeType = 'application/json';
      break;
    case 'csv':
      content = await generateCSV(results);
      mimeType = 'text/csv';
      break;
    default:
      throw new InvalidFormatError(format);
  }

  // 2. Сохранение
  const exportId = generateExportId();
  const url = await uploadExport(exportId, content, mimeType);

  // 3. Создание записи
  const exportEntry: ExportEntry = {
    format,
    url,
    createdAt: Date.now(),
    expiresAt: Date.now() + 7 * 24 * 60 * 60 * 1000 // 7 дней
  }
```

```

};

await saveExportEntry(combinedId, exportEntry);

return exportEntry;
}

async function generatePDF(
  combined: CombinedAttachment,
  results: CombinedResults
): Promise<Buffer> {
  const doc = new PDFDocument();

  // Заголовок
  doc.fontSize(20).text(combined.preset, { align: 'center' });
  doc.moveDown();

  // Компоненты
  for (const component of combined.components) {
    doc.fontSize(14).text(getComponentTitle(component));
    doc.moveDown(0.5);

    // Рендер компонента
    await renderComponentToPDF(doc, component,
results.componentResults[component.id]);

    doc.moveDown();
  }

  // Результаты
  doc.addPage();
  doc.fontSize(16).text('Результаты', { underline: true });
  doc.moveDown();

  doc.fontSize(12).text(`Всего взаимодействий:
${results.summary.totalInteractions}`);
  doc.text(`Участников: ${results.summary.uniqueParticipants}`);
  doc.text(`Завершение: ${(results.summary.completionRate *
100).toFixed(1)}%`);

  return doc;
}

```

5. UI/UX SPECIFICATION

5.1 Планировщик событий

 Планировщик: Корпоратив 2026
  15 участников

Описание

Планируем ежегодный корпоратив компании



Предложенные даты:

29 июня • 30 июня • 1 июля



Предложенные места:

Ресторан Пушкин • Лофт Центр • Загородный клуб



Выберите дату

15 голосов

- | | | | |
|--|-----|-------------|--|
| ☐ 29 июня | 20% | <div></div> | |
| ☒ 30 июня | 53% | <div></div> |  Вы |
| ☐ 1 июля | 27% | <div></div> | |



Выберите место

14 голосов

- | | | |
|--|-----|-------------|
| ☒ Ресторан Пушкин | 57% | <div></div> |
| ☐ Лофт Центр | 29% | <div></div> |
| ☐ Загородный клуб | 14% | <div></div> |



План мероприятия

50% выполнено

- | | |
|---|---------|
| ☒ Сбор гостей | (12/15) |
| ☒ Ужин | (10/15) |
| ☐ Конкурсы | (0/15) |
| ☐ Награждение | (0/15) |



15 участников • 87% завершение • Завершится через 2д



Обсудить



Экспорт



Полная статистика

5.2 Презентация



Презентация: Отчёт Q2 2026

◀ 1/5 ▶

ОТЧЁТ Q2 2026

Balloo Messenger

[Логотип]

Июнь 2026

● ○ ○ ○ ○

[← Назад] [Вперёд →] [📄 Скачать PDF]

5.3 Трекер целей

🎯 Цель: Запуск Premium функций 60% завершено

📄 Описание

Запустить 5 premium вложений для пользователей со статусом Шейх до конца Q3 2026

📊 Прогресс

60% (3/5)

- ☒ Переводы (Payment)
- ☒ Игры (Game)
- ☒ Исчезающие (Expiring)
- ☐ Совместные (Collaborative)
- ☐ Комбинированные (Combined)

📝 Заметки

- Переводы готовы к запуску 15 июня
- Игры на тестировании
- Нужно ускорить разработку Collaborative

6. МЕТРИКИ

```
interface CombinedMetrics {  
    // Создание  
    totalCombined: number;  
    createdToday: number;  
    byPreset: Record<CombinedPreset, number>;  
  
    // Взаимодействия  
    totalInteractions: number;  
    interactionsToday: number;  
    averageInteractionsPerCombined: number;  
  
    // Завершение  
    completionRate: number;  
    averageCompletionTime: number;  
  
    // Участники  
    uniqueParticipants: number;  
    averageParticipantsPerCombined: number;  
  
    // Экспорт  
    totalExports: number;  
    byFormat: Record<string, number>;  
  
    // Шаблоны  
    templatesCreated: number;  
    templateUsage: number;  
    topTemplates: { id: string; usage: number }[];  
}
```

 **Статус документа:** Complete

 **Baloo - Проверни общение!**